EXTENSE LLC

XML Fundamentals Training Guide

From XML Basics to Your First DITA Project

Comprehensive Training Guide

Contents

Preface

How to use this training guide, what you will learn, and who this guide is for.

Who Is This Guide For?

This training guide is designed for anyone who needs to understand XML technologies — whether you are a technical writer, content strategist, developer, or information architect. No prior XML experience is required.

By the end of this guide, you will have the foundational knowledge needed to work with XML-based standards like DITA, XSLT, and other markup languages.

What You Will Learn

The guide is organized into four progressive parts:

1. **XML Fundamentals** — Syntax rules, elements, attributes, namespaces, and the document tree. These are the building blocks of every XML technology.
2. **Document Type Definitions (DTD)** — Your first schema language. Learn how DTDs define structure, enforce validation, and manage reusable content with entities.
3. **XML Schema Definition (XSD)** — A more powerful schema language with data types, restrictions, and type inheritance.
4. **Introduction to DITA** — How all of these concepts come together in the Darwin Information Typing Architecture — the industry standard for structured content.

How to Use This Guide

Each chapter builds on the previous one. We recommend working through the parts in order, especially if you are new to XML. Each chapter includes:

- **Explanatory text** with real-world context and examples
- **Code examples** you can type and experiment with
- **Quick-reference topics** you can return to while working

The **Learner Exercises** section provides hands-on practice for each part. Complete the exercises after reading the corresponding chapters — they reinforce the concepts through a progressive project.

> **Tip**
> Keep the quick-reference topics open while doing the exercises. They serve as a concise syntax guide you will use often as you work.

Conventions Used

- `Monospace text` indicates code, element names, attribute names, or file names.
- **Bold text** indicates key terms or important concepts.
- `File paths` indicate files or directories you should create or navigate to.

- Notes labeled **Tip**, **Important**, or **Note** provide supplementary guidance.

Part

I

XML Fundamentals

Topics:

What Is XML?

XML (Extensible Markup Language) is a text-based format for representing structured data — designed to be both human-readable and machine-processable.

What does XML stand for and why does it exist?

XML stands for **Extensible Markup Language**. It was developed by the World Wide Web Consortium (W3C) and became a recommendation in 1998. XML was created to solve a fundamental problem: how to represent structured data in a way that is:

- **Self-describing:** The data carries its own labels (element names), so you can understand what each piece means without external documentation.
- **Platform-independent:** XML is plain text — any programming language, operating system, or tool can read and write it.
- **Extensible:** Unlike HTML, which has a fixed set of tags, XML lets you define your own elements and attributes for any domain.
- **Validatable:** You can define rules (DTD, XSD) that specify what structure is allowed, and tools can check compliance automatically.

XML is not a programming language and it does not do anything by itself. It is a format for carrying data between systems, storing configuration, and defining document structures.

How is XML different from HTML?

HTML and XML are both markup languages with angle-bracket syntax, but they serve fundamentally different purposes:

Feature	HTML	XML
Purpose	Display content in web browsers	Store and transport structured data
Tag set	Fixed set of predefined tags (`<div>`, `<p>`, `<h1>`)	You define your own tags (`<invoice>`, `<patient>`)
Strictness	Browsers forgive errors — unclosed tags, missing quotes	Parsers reject malformed documents — strict syntax required
Semantics	Tags describe presentation (`<b>` = bold)	Tags describe meaning (`<price>` = a price value)
Case sensitivity	Case-insensitive (`<DIV>` = `<div>`)	Case-sensitive (`<Item>` # `<item>`)

Tip

Think of HTML as a language for **showing** data to humans, and XML as a language for **carrying** data between systems.

Where is XML used in the real world?

XML is everywhere — even if you do not see it directly:

- **Configuration files:** Maven (`pom.xml`), Android (`AndroidManifest.xml`), Spring Framework, and many build tools use XML for configuration.
- **Document formats:** Microsoft Office (.docx, .xlsx, .pptx) are ZIP archives containing XML files. LibreOffice uses ODF, also XML-based.
- **Data exchange:** SOAP web services, banking (ISO 20022), healthcare (HL7 CDA), and government filing systems use XML for data interchange.
- **Publishing:** DITA, DocBook, and JATS (scientific journals) are XML-based standards for structured documentation.
- **Web technologies:** SVG (vector graphics), RSS/Atom (feeds), XHTML, and XSLT (transformations) are all XML applications.

Your first look at XML

Here is a minimal XML document:

```
<?xml version="1.0" encoding="UTF-8"?>
<bookstore>
  <book category="programming">
    <title>Learning XML</title>
    <author>Jane Smith</author>
    <year>2024</year>
    <price currency="USD">29.95</price>
  </book>
  <book category="fiction">
    <title>The Great Adventure</title>
    <author>John Doe</author>
    <year>2023</year>
    <price currency="USD">14.99</price>
  </book>
</bookstore>
```

Notice how every piece of data is labeled by elements (`<title>`, `<author>`) and qualified by attributes (`category="programming"`, `currency="USD"`). The structure is a tree — `<bookstore>` contains `<book>` elements, which contain child elements.

Why learn XML today?

Despite the rise of JSON for web APIs, XML remains essential because:

- **Schema validation** — XML has mature, powerful schema languages (DTD, XSD, RELAX NG) that JSON Schema cannot match for complex document structures.
- **Document processing** — XSLT and XPath provide a complete transformation ecosystem that has no equivalent in JSON.
- **Industry standards** — DITA, DocBook, HL7, XBRL, and dozens of critical industry standards are XML-based and will remain so for decades.
- **Mixed content** — XML handles text intermixed with markup naturally (`<p>Click <b>Save</b> to continue.</p>`), while JSON cannot.

XML Syntax Rules: Well-Formedness

Every XML document must follow strict syntax rules to be considered well-formed — a parser will reject any document that violates them.

What does "well-formed" mean?

A well-formed XML document is one that follows all the syntax rules of the XML specification. This is the **minimum requirement** for any XML parser to accept a document. A document that is not well-formed is not XML at all — parsers will refuse to process it.

Well-formedness is different from **validity**. A well-formed document has correct syntax; a valid document also conforms to a schema (DTD or XSD). You will learn about validity in later chapters.

Rule 1: The XML declaration

Every XML document should begin with an XML declaration:

```
<?xml version="1.0" encoding="UTF-8"?>
```

- `version="1.0"` — The XML version. Almost all XML in production uses version 1.0.
- `encoding="UTF-8"` — The character encoding. UTF-8 is the default and recommended encoding.

> **Important**
> The XML declaration must be the very first thing in the file — no blank lines, no spaces, no comments before it.

Rule 2: Exactly one root element

Every XML document must have exactly one root element that contains all other elements:

```
<!-- CORRECT: single root element -->
<catalog>
  <product>Widget</product>
  <product>Gadget</product>
</catalog>

<!-- WRONG: two root elements -->
<product>Widget</product>
<product>Gadget</product>
```

Rule 3: Every element must be closed

Unlike HTML, XML requires every opening tag to have a matching closing tag:

```
<!-- CORRECT -->
<paragraph>Hello, world.</paragraph>

<!-- WRONG: unclosed element -->
```

```
<paragraph>Hello, world.
```

Empty elements (elements with no content) can be written in two ways:

```
<!-- Both are valid -->
<linebreak></linebreak>
<linebreak/>
```

Rule 4: Elements must be properly nested

Child elements must be completely contained within their parent. Overlapping tags are not allowed:

```
<!-- CORRECT: proper nesting -->
<bold><italic>important text</italic></bold>

<!-- WRONG: overlapping elements -->
<bold><italic>important text</bold></italic>
```

Rule 5: XML is case-sensitive

Element and attribute names are case-sensitive. The opening and closing tags must use identical casing:

```
<!-- CORRECT -->
<Title>My Book</Title>

<!-- WRONG: case mismatch -->
<Title>My Book</title>
```

> **Tip**
> Convention: Use lowercase for element names and attributes in your own XML vocabularies. This is consistent with DITA, HTML5, and most modern XML standards.

Rule 6: Attribute values must be quoted

All attribute values must be enclosed in quotes — either single or double:

```
<!-- CORRECT -->
<book category="fiction">
<book category='fiction'>

<!-- WRONG: unquoted attribute -->
<book category=fiction>
```

Additional attribute rules:

- An element cannot have two attributes with the same name.
- Attribute values cannot contain the `<` character directly — use `<` instead.

Rule 7: Special characters require entity references

Five characters have special meaning in XML and must be escaped when they appear in text content or attribute values:

Character	Entity Reference	Meaning
`<`	`<`	Less-than — starts a tag
`>`	`>`	Greater-than — ends a tag
`&`	`&`	Ampersand — starts an entity reference
`"`	`&quot;`	Double quote — used in attribute values
`'`	`&apos;`	Apostrophe — used in attribute values

Comments and processing instructions

XML supports comments and processing instructions:

```
<!-- This is a comment. It is ignored by parsers. -->

<?xml-stylesheet type="text/xsl" href="transform.xsl"?>
```

Comments cannot contain the double-hyphen sequence (`--`) inside the comment body, and they cannot be nested.

CDATA sections: Escaping blocks of text

When you need to include a large block of text that contains many special characters (such as code examples), use a CDATA section:

```
<script><![CDATA[
  if (x < 10 && y > 5) {
    alert("Valid range");
  }
]]></script>
```

Everything inside `<![CDATA[` ... `]]>` is treated as plain text — no entity escaping is needed.

Common well-formedness errors

When you are starting out, watch for these frequent mistakes:

1. Forgetting to close a tag
2. Mismatched case between opening and closing tags
3. Using `&` without escaping it as `&`

4. Putting two root elements in one document
5. Missing quotes around attribute values

> **Tip**
> Use an XML-aware editor (VS Code with the XML extension, or Oxygen XML) — it will highlight well-formedness errors in real time as you type.

Elements, Attributes, and the XML Tree

XML documents are built from elements and attributes that form a hierarchical tree structure — understanding this tree is the key to working with XML effectively.

Elements: The building blocks of XML

An element is the fundamental unit of XML. It consists of a start tag, content, and an end tag:

```
<title>Learning XML</title>
  #         #             #
start tag  content      end tag
```

Elements can contain:

- **Text content:** `<name>Jane Smith</name>`
- **Child elements:** `<address><city>Austin</city></address>`
- **Mixed content:** `<p>Click <b>Save</b> to confirm.</p>`
- **Nothing (empty):** `<linebreak/>`

Element naming rules

XML element names must follow these rules:

- Must start with a letter or underscore (`_`)
- Can contain letters, digits, hyphens, underscores, and periods
- Cannot start with the letters `xml` (reserved)
- Cannot contain spaces
- Are case-sensitive

```
<!-- VALID element names -->
<firstName>  <first-name>  <first_name>  <_internal>

<!-- INVALID element names -->
<1stName>    <first name>  <xml-data>
```

> **Tip**
> Choose a consistent naming convention for your XML vocabulary. Common styles: `camelCase`, `kebab-case`, or `snake_case`. DITA uses kebab-case for file names and camelCase for some attribute values.

Attributes: Metadata on elements

Attributes provide additional information about an element. They appear inside the start tag as name-value pairs:

```
<book isbn="978-0-13-468599-1" format="hardcover">
  <title>The Pragmatic Programmer</title>
</book>
```

Key rules for attributes:

- Values must be quoted (single or double quotes)
- An element cannot have duplicate attribute names
- Attribute order does not matter
- Attributes cannot contain child elements — they hold simple values only

When to use elements vs. attributes

A common design question: should data be an element or an attribute? Here are guidelines:

Use an element when...	Use an attribute when...
The data has sub-structure	The data is a simple value (ID, type, status)
The data might contain markup	The data qualifies or classifies the element
The data can repeat (multiple values)	The data appears at most once per element
The data is the primary content	The data is metadata about the content

Tip

When in doubt, use elements. They are more flexible and can be extended later without breaking existing documents.

The XML document tree

Every XML document forms a tree (also called the **document object model** or **DOM**). Understanding the tree is essential for navigation using XPath and for understanding schema validation.

```
<library>
  <section name="fiction">
    <book>
      <title>Dune</title>
      <author>Frank Herbert</author>
    </book>
    <book>
      <title>Neuromancer</title>
      <author>William Gibson</author>
    </book>
  </section>
```

```
</library>
```

The tree relationships:

- `<library>` is the **root** element
- `<section>` is a **child** of `<library>`
- `<library>` is the **parent** of `<section>`
- The two `<book>` elements are **siblings**
- `<title>` and `<author>` are **children** of `<book>`
- `<title>` is a **descendant** of `<library>`

These terms — root, parent, child, sibling, descendant, ancestor — are the vocabulary you will use throughout XML, XPath, XSLT, and DITA.

A preview: XML Namespaces

When different XML vocabularies are combined in one document, names can collide. XML Namespaces solve this by prefixing element names with a URI-based identifier:

```
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:svg="http://www.w3.org/2000/svg">
  <body>
    <svg:svg width="100" height="100">
      <svg:circle cx="50" cy="50" r="40"/>
    </svg:svg>
  </body>
</html>
```

You will encounter namespaces when working with XSD schemas (which use the `xs:` or `xsd:` prefix) and when combining multiple XML standards in one document.

XML Namespaces

Namespaces prevent naming conflicts when XML vocabularies from different sources are combined in a single document by qualifying element and attribute names with a unique URI.

Why do we need namespaces?

Imagine two XML vocabularies that both define a `<title>` element — one for books and one for job titles. When these vocabularies appear in the same document, a parser cannot tell which `<title>` is which. Namespaces solve this by giving each vocabulary a unique identifier.

```
<!-- Without namespaces: ambiguous -->
<record>
  <title>Learning XML</title>       <!-- Book title? -->
  <title>Senior Engineer</title>    <!-- Job title? -->
</record>
```

Declaring namespaces

A namespace is declared using the `xmlns` attribute. There are two forms:

Default namespace — applies to the element and all descendants without a prefix:

```
<bookstore xmlns="http://example.com/books">
  <title>Learning XML</title>  <!-- in the books namespace -->
</bookstore>
```

Prefixed namespace — uses a short prefix to identify elements from a specific vocabulary:

```
<record xmlns:bk="http://example.com/books"
        xmlns:hr="http://example.com/hr">
  <bk:title>Learning XML</bk:title>       <!-- Book title -->
  <hr:title>Senior Engineer</hr:title>    <!-- Job title -->
</record>
```

Important

The namespace URI is just an identifier — it does not need to point to an actual web page. By convention, organizations use their domain name to ensure uniqueness.

Namespace scope

A namespace declaration is in scope for the element where it is declared and all of that element's descendants. You can override a namespace on a child element:

```
<root xmlns="http://example.com/default">
  <child>In default namespace</child>
  <other xmlns="http://example.com/other">
    <child>In other namespace</child>
  </other>
</root>
```

Common namespaces you will encounter

Prefix	Namespace URI	Used for
`xs:` or `xsd:`	http://www.w3.org/2001/XMLSchema	XML Schema Definition (XSD)
`xsl:`	http://www.w3.org/1999/XSL/Transform	XSLT transformations
`xhtml:`	http://www.w3.org/1999/xhtml	XHTML documents
`svg:`	http://www.w3.org/2000/svg	SVG vector graphics

Namespaces in DITA

DITA topics typically do not use namespace prefixes because DITA has its own vocabulary that does not collide with other schemas. However, you will encounter namespaces when:

- Writing XSLT stylesheets that transform DITA content
- Working with XSD-based DITA grammar files (DITA 1.3+ uses RELAX NG and XSD grammars)
- Embedding foreign content (such as MathML or SVG) within DITA topics

XML Syntax Quick Reference

A compact reference card for XML syntax, entity references, naming rules, and common patterns — keep this open while writing your first XML documents.

Document structure template

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Optional: DOCTYPE declaration for DTD validation -->
<!DOCTYPE rootElement SYSTEM "schema.dtd">

<rootElement attribute="value">
  <childElement>Text content</childElement>
  <emptyElement/>
  <!-- Comment -->
  <![CDATA[Unescaped text with < and & characters]]>
</rootElement>
```

Built-in entity references

Character	Entity	Usage
`<`	`<`	Required in text and attributes
`>`	`>`	Recommended in text for clarity
`&`	`&`	Required in text and attributes
`"`	`&quot;`	Required in double-quoted attributes
`'`	`&apos;`	Required in single-quoted attributes

Element and attribute naming rules

Rule	Valid	Invalid
Must start with letter or _	`item`, `_temp`	`1item`, `-item`

Rule	Valid	Invalid
Can contain letters, digits, -, _, .	`item-2`, `item.v1`	`item 2`, `item@v1`
Cannot start with "xml"	`myxml`	`xmlData`
Case-sensitive	`Item` # `item`	N/A

Well-formedness checklist

- # XML declaration at the very start of the file
- # Exactly one root element
- # Every start tag has a matching end tag (or is self-closing)
- # Elements are properly nested (no overlapping)
- # Attribute values are quoted
- # No duplicate attributes on the same element
- # Special characters are escaped with entity references
- # Element names follow naming rules

Part

II

Document Type Definitions (DTD)

Topics:

What Is a DTD?

A Document Type Definition (DTD) is the original XML schema language — it defines the allowed structure of an XML document by declaring which elements, attributes, and entities are permitted.

Why do we need schemas?

A well-formed XML document has correct syntax, but that says nothing about whether its **content structure** makes sense. Consider:

```
<!-- Well-formed, but is it correct? -->
<book>
  <price>Learning XML</price>     <!-- Title in price? -->
  <author>29.95</author>          <!-- Number as author? -->
</book>
```

A schema defines the **rules** for what a valid document looks like: which elements can appear, in what order, how many times, and what attributes they can carry. A **validating parser** checks a document against its schema and reports any violations.

A DTD (Document Type Definition) is the oldest and simplest schema language for XML. It has been part of XML since the very beginning.

Well-formed vs. valid

Well-formed	Valid
Follows XML syntax rules	Follows XML syntax rules AND conforms to a schema
Checked by any XML parser	Checked by a validating parser with a schema
Required for all XML	Optional — but strongly recommended for production content

All valid documents are well-formed, but not all well-formed documents are valid.

Internal vs. external DTDs

A DTD can be declared in two places:

Internal DTD — embedded directly in the XML document:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note [
  <!ELEMENT note (to, from, subject, body)>
  <!ELEMENT to (#PCDATA)>
  <!ELEMENT from (#PCDATA)>
  <!ELEMENT subject (#PCDATA)>
  <!ELEMENT body (#PCDATA)>
```

```
]>
<note>
  <to>Student</to>
  <from>Instructor</from>
  <subject>Welcome</subject>
  <body>Welcome to XML training!</body>
</note>
```

External DTD — stored in a separate file and referenced from the document:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE note SYSTEM "note.dtd">
<note>
  <to>Student</to>
  <from>Instructor</from>
  <subject>Welcome</subject>
  <body>Welcome to XML training!</body>
</note>
```

In practice, external DTDs are much more common because they can be shared across multiple documents. DITA uses external DTDs — every DITA topic begins with a DOCTYPE declaration that references the DITA DTD files.

SYSTEM vs. PUBLIC identifiers

External DTDs use two forms of reference:

- **SYSTEM** — points to a file by URI (path or URL): `<!DOCTYPE note SYSTEM "note.dtd">`
- **PUBLIC** — uses a formal public identifier (FPI) plus a fallback URI. This is how DITA DTDs are referenced:

```
<!DOCTYPE concept PUBLIC "-//OASIS//DTD DITA Concept//EN" "concept.dtd">
```

The PUBLIC identifier (`-//OASIS//DTD DITA Concept//EN`) allows XML catalogs to resolve the DTD location without depending on a specific file path.

What can a DTD define?

A DTD can declare four types of constructs:

1. **Elements** — which elements exist and what they can contain
2. **Attributes** — which attributes each element can have, their types, and default values
3. **Entities** — reusable text shortcuts and character references
4. **Notations** — identifiers for non-XML data formats (rarely used today)

The following chapters explore element and attribute declarations in detail.

DTD Element Declarations

Element declarations in a DTD define which elements can appear in a document and specify their allowed content — child elements, text, or a combination of both.

Element declaration syntax

The basic syntax for declaring an element is:

```
<!ELEMENT element-name (content-model)>
```

The content model specifies what the element can contain. There are several types:

Text-only elements (#PCDATA)

An element that contains only text (no child elements) uses `#PCDATA` (Parsed Character Data):

```
<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
<!ELEMENT price (#PCDATA)>
```

Usage in an XML document:

```
<title>Learning XML</title>
<price>29.95</price>
```

Important

`#PCDATA` stands for "Parsed Character Data" — meaning the parser will process entity references like `&` inside the text. It does **not** mean the element can only contain plain text strings.

Elements with child elements (sequences)

An element that must contain specific child elements in a specific order uses a comma-separated list:

```
<!ELEMENT book (title, author, year, price)>
```

This means a `<book>` must contain exactly one `<title>`, followed by one `<author>`, then one `<year>`, then one `<price>` — in that exact order.

Occurrence indicators: How many times?

By default, a child element must appear exactly once. You can change this with occurrence indicators:

Indicator	Meaning	Example
(none)	Exactly one (required)	`title` — one title required
`?`	Zero or one (optional)	`subtitle?` — subtitle is optional
`*`	Zero or more	`author*` — any number of authors (including none)
`+`	One or more (at least one required)	`chapter+` — at least one chapter required

```
<!-- A book has one title, optional subtitle, one or more authors, and one price -->
<!ELEMENT book (title, subtitle?, author+, price)>
```

Choice groups (alternatives)

Use the pipe character (`|`) to indicate that one of several elements can appear:

```
<!-- A payment is EITHER a credit-card OR a bank-transfer OR cash -->
<!ELEMENT payment (credit-card | bank-transfer | cash)>
```

You can combine sequences and choices with parentheses:

```
<!-- A product has a name, then either a description or a summary, then a price -->
<!ELEMENT product (name, (description | summary), price)>
```

Mixed content

An element with mixed content can contain both text and child elements. The syntax uses `#PCDATA` with choices and the `*` indicator:

```
<!ELEMENT paragraph (#PCDATA | bold | italic | link)*>
```

This allows content like:

```
<paragraph>Click <bold>Save</bold> to store your <italic>document</italic>.</paragraph>
```

Important

In mixed content declarations, `#PCDATA` must be listed **first**, and the group must use choice (`|`) with `*`. You cannot use sequences with mixed content.

EMPTY and ANY content

Two special content models:

```
<!-- Element must have no content -->
<!ELEMENT linebreak EMPTY>

<!-- Element can contain anything (text, elements, or both) -->
<!ELEMENT container ANY>
```

`EMPTY` is common for elements like line breaks or images that carry information through attributes only. `ANY` is rarely used in production because it provides no validation.

Complete example: A bookstore DTD

```
<!ELEMENT bookstore (book+)>
<!ELEMENT book (title, author+, year, price, description?)>
<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
<!ELEMENT year (#PCDATA)>
<!ELEMENT price (#PCDATA)>
<!ELEMENT description (#PCDATA | bold | italic)*>
<!ELEMENT bold (#PCDATA)>
<!ELEMENT italic (#PCDATA)>
```

This DTD says:

- A `<bookstore>` contains one or more `<book>` elements
- A `<book>` has a required title, one or more authors, a year, a price, and an optional description
- The `<description>` can contain mixed content with bold and italic markup

DTD Attribute Declarations and Entities

Attribute declarations in a DTD specify which attributes each element can carry, their data types, and whether they are required or optional — while entities provide reusable content shortcuts.

Attribute declaration syntax

Attributes are declared using `<!ATTLIST>`:

```
<!ATTLIST element-name
    attribute-name  type  default-behavior
>
```

You can declare multiple attributes for the same element in a single ATTLIST declaration:

```
<!ATTLIST book
    isbn      CDATA     #REQUIRED
    category  CDATA     #IMPLIED
    format    (hardcover | paperback | ebook)  "paperback"
```

```
>
```

Attribute types

Type	Meaning	Example
`CDATA`	Any character data (free text)	`title CDATA #REQUIRED`
`ID`	Unique identifier within the document	`id ID #REQUIRED`
`IDREF`	Reference to an ID in the same document	`authorRef IDREF #IMPLIED`
`IDREFS`	Space-separated list of ID references	`relatedTo IDREFS #IMPLIED`
Enumeration	One value from a fixed list	`(small | medium | large)`
`NMTOKEN`	A name token (letters, digits, hyphens, underscores, periods)	`status NMTOKEN #IMPLIED`

Default behaviors

Default	Meaning
`#REQUIRED`	The attribute must be present in every instance of the element
`#IMPLIED`	The attribute is optional — no default value if omitted
`#FIXED "value"`	The attribute always has this value (cannot be changed by the author)
`"value"`	Default value used when the attribute is not specified

Complete attribute example

```
<!-- DTD declarations -->
<!ELEMENT product (name, description?)>
<!ATTLIST product
    id        ID                              #REQUIRED
    category  (electronics | clothing | food)  #REQUIRED
    status    (active | discontinued)          "active"
    sku       CDATA                           #IMPLIED
>

<!-- Valid XML instance -->
<product id="p001" category="electronics" sku="EL-1234">
  <name>Wireless Mouse</name>
  <description>Ergonomic design with long battery life.</description>
```

```
</product>
```

In this example, `status` defaults to `"active"` because it was not specified in the XML.

General entities: Reusable text shortcuts

A general entity defines a reusable text fragment that can be referenced by name in XML documents:

```
<!-- DTD -->
<!ENTITY company-name "Acme Corporation">
<!ENTITY copyright "Copyright 2026 Acme Corporation. All rights reserved.">

<!-- Used in XML -->
<footer>&copyright;</footer>
<publisher>&company-name;</publisher>
```

When the parser processes the document, it replaces `&company-name;` with the text `Acme Corporation`.

Parameter entities: Reusable DTD fragments

Parameter entities are used **within DTDs** (not in XML documents) to factor out repeated declarations. They use `%` instead of `&`:

```
<!-- Define a reusable content model -->
<!ENTITY % inline-content "(#PCDATA | bold | italic | code)*">

<!-- Use it in multiple element declarations -->
<!ELEMENT paragraph %inline-content;>
<!ELEMENT note %inline-content;>
<!ELEMENT warning %inline-content;>
```

DITA's DTD files make extensive use of parameter entities for modularity and specialization. This is how DITA allows new topic types to extend the base content models.

External entities

External entities reference content from another file:

```
<!-- Include content from an external file -->
<!ENTITY chapter1 SYSTEM "chapter1.xml">

<!-- Used in the document -->
<book>
  &chapter1;
</book>
```

Warning

External entities can be a security risk (XML External Entity attacks — XXE). Modern XML processors often disable external entity resolution by default. When processing untrusted XML, always disable external entity loading.

Limitations of DTDs

DTDs have served XML well since 1998, but they have significant limitations that led to the creation of XML Schema (XSD):

- **No data types:** All text content is `#PCDATA` — you cannot specify that an element must contain a number, date, or email address.
- **No namespace support:** DTDs were designed before XML namespaces existed and have no way to validate namespace-qualified documents.
- **Non-XML syntax:** DTD declarations use their own syntax, not XML. This means you cannot use XML tools to process DTDs.
- **Limited occurrence control:** You can say "one or more" (`+`) but not "between 2 and 5".
- **No context-sensitive rules:** An element always has the same content model regardless of where it appears in the tree.

Despite these limitations, DTDs remain widely used — DITA 1.x uses DTDs as its primary grammar, and many legacy XML systems depend on them.

DTD Syntax Quick Reference

A compact reference for DTD element declarations, attribute types, entities, and content model patterns — use this while writing and reading DTD files.

Element declaration patterns

Pattern	Declaration	Meaning
Text only	`<!ELEMENT name (#PCDATA)>`	Contains text, no children
Sequence	`<!ELEMENT x (a, b, c)>`	a then b then c, each once
Choice	`<!ELEMENT x (a | b | c)>`	Exactly one of a, b, or c
Optional	`<!ELEMENT x (a, b?)>`	a required, b optional
One or more	`<!ELEMENT x (a+)>`	At least one a
Zero or more	`<!ELEMENT x (a*)>`	Any number of a (including none)
Mixed content	`<!ELEMENT x (#PCDATA | a | b)*>`	Text mixed with a and b elements
Empty	`<!ELEMENT x EMPTY>`	No content allowed
Any	`<!ELEMENT x ANY>`	Any content allowed

Attribute types and defaults

Type	Description
`CDATA`	Free text (character data)
`ID`	Unique identifier in the document
`IDREF`	Reference to an ID value
`IDREFS`	Space-separated list of ID references
`NMTOKEN`	Name token (restricted character set)
`(a|b|c)`	Enumerated values — must be one of the listed options

Default	Description
`#REQUIRED`	Must be provided by the author
`#IMPLIED`	Optional, no default
`#FIXED "val"`	Always this value — cannot be changed
`"val"`	Default value if attribute is omitted

Entity patterns

Type	Declaration	Usage
General entity	`<!ENTITY name "text">`	`&name;` in XML
Parameter entity	`<!ENTITY % name "text">`	`%name;` in DTD
External entity	`<!ENTITY name SYSTEM "file.xml">`	`&name;` in XML

Complete DTD template

```
<!-- bookstore.dtd -->
<!ENTITY company "My Publishing House">

<!ELEMENT bookstore (book+)>

<!ELEMENT book (title, author+, year, price, description?)>
<!ATTLIST book
    isbn      CDATA  #REQUIRED
    category  (fiction | non-fiction | technical)  #REQUIRED
    format    (hardcover | paperback | ebook)  "paperback"
>

<!ELEMENT title (#PCDATA)>
<!ELEMENT author (#PCDATA)>
<!ELEMENT year (#PCDATA)>
<!ELEMENT price (#PCDATA)>
```

```
<!ATTLIST price currency CDATA "USD">
<!ELEMENT description (#PCDATA | bold | italic)*>
<!ELEMENT bold (#PCDATA)>
<!ELEMENT italic (#PCDATA)>
```

Part

XML Schema Definition (XSD)

Topics:

What Is XML Schema (XSD)?

XML Schema Definition (XSD) is a powerful schema language that describes the structure and data types of XML documents using XML syntax itself — offering far more precision than DTDs.

Why was XSD created?

DTDs, while useful, have significant limitations. The W3C created XML Schema (commonly called XSD because schema files use the `.xsd` extension) to address them. XSD became a W3C recommendation in 2001 and is one of the two most widely used schema languages today (alongside RELAX NG).

The key motivations for XSD were:

- **Data types:** DTDs treat all content as text. XSD provides 44 built-in data types — integers, dates, booleans, URIs, and more.
- **Namespace support:** XSD was designed for namespace-aware validation from the start.
- **XML syntax:** XSD schemas are themselves XML documents, so you can create, parse, and transform them with standard XML tools.
- **Fine-grained constraints:** XSD supports minimum/maximum occurrences, string length limits, pattern matching, and value ranges.

DTD vs. XSD comparison

Feature	DTD	XSD
Syntax	Non-XML (own declaration syntax)	XML (schemas are XML documents)
Data types	Only CDATA, ID, IDREF, NMTOKEN, enumerations	44+ built-in types: string, integer, date, boolean, anyURI, etc.
Occurrence control	`?` `*` `+` only	`minOccurs`/`maxOccurs` with any number
Namespace support	None	Full namespace-aware validation
Inheritance	Not supported	Complex type extension and restriction
Pattern constraints	Not supported	Regular expression patterns on values
Learning curve	Simple — can learn in an hour	Moderate — richer but more verbose

Structure of an XSD file

An XSD file is an XML document with `<xs:schema>` as its root element:

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- Element declarations -->
  <xs:element name="bookstore">
    <!-- type definition here -->
  </xs:element>

</xs:schema>
```

The `xs:` prefix is bound to the W3C XML Schema namespace. Some schemas use `xsd:` instead — both are equivalent.

How to link an XML document to its XSD

Unlike DTDs (which use `<!DOCTYPE>`), XSD references are specified using attributes on the root element:

```
<bookstore xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
           xsi:noNamespaceSchemaLocation="bookstore.xsd">
  <!-- content -->
</bookstore>
```

If the document uses a target namespace:

```
<bookstore xmlns="http://example.com/bookstore"
           xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
           xsi:schemaLocation="http://example.com/bookstore bookstore.xsd">
  <!-- content -->
</bookstore>
```

When to use DTD vs. XSD

- **Use DTD** when working with legacy systems, DITA 1.x content, or when you need the simplest possible validation.
- **Use XSD** when you need data type validation, namespace support, precise occurrence constraints, or when your schema needs to be processed as XML.
- **Either works** when you only need basic structure validation (element order and nesting).

> **Tip**
> DITA provides both DTD and XSD grammar files. You can validate DITA content against either format — the choice depends on your toolchain's preferences.

XSD Elements and Data Types

XSD defines elements using simple types (for text-only content) and complex types (for elements with children or attributes) — with 44 built-in data types for precise validation.

Simple elements

A simple element contains only text — no child elements and no attributes. You declare it using a built-in type:

```
<xs:element name="title" type="xs:string"/>
<xs:element name="year" type="xs:integer"/>
<xs:element name="price" type="xs:decimal"/>
<xs:element name="published" type="xs:date"/>
<xs:element name="inStock" type="xs:boolean"/>
```

These declarations say: `<title>` must contain a string, `<year>` must contain an integer, `<price>` must contain a decimal number, and so on. A document with `<year>not-a-number</year>` would fail validation.

Common built-in data types

Type	Description	Example value
`xs:string`	Any character string	Hello World
`xs:integer`	Whole number (positive or negative)	42
`xs:decimal`	Decimal number	29.95
`xs:boolean`	true/false or 1/0	true
`xs:date`	Calendar date (YYYY-MM-DD)	2026-04-05
`xs:dateTime`	Date and time	2026-04-05T14:30:00
`xs:anyURI`	Uniform Resource Identifier	https://example.com
`xs:positiveInteger`	Integer greater than zero	7
`xs:ID`	Unique identifier (like DTD ID)	item-001
`xs:NMTOKEN`	Name token (no spaces, restricted chars)	draft

Default and fixed values

Simple elements can have default or fixed values:

```
<!-- If omitted, defaults to "Unknown" -->
<xs:element name="author" type="xs:string" default="Unknown"/>

<!-- Always this value – cannot be changed -->
<xs:element name="version" type="xs:string" fixed="1.0"/>
```

Complex types: Elements with structure

An element that contains child elements, attributes, or both uses a complex type. There are several patterns:

Sequence — children must appear in the specified order:

```
<xs:element name="book">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="title" type="xs:string"/>
      <xs:element name="author" type="xs:string"/>
      <xs:element name="year" type="xs:integer"/>
      <xs:element name="price" type="xs:decimal"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Choice — exactly one of the listed children can appear:

```
<xs:element name="payment">
  <xs:complexType>
    <xs:choice>
      <xs:element name="creditCard" type="xs:string"/>
      <xs:element name="bankTransfer" type="xs:string"/>
      <xs:element name="cash" type="xs:string"/>
    </xs:choice>
  </xs:complexType>
</xs:element>
```

All — children can appear in any order, each at most once:

```
<xs:element name="address">
  <xs:complexType>
    <xs:all>
      <xs:element name="street" type="xs:string"/>
      <xs:element name="city" type="xs:string"/>
      <xs:element name="zip" type="xs:string"/>
    </xs:all>
  </xs:complexType>
</xs:element>
```

Occurrence constraints: minOccurs and maxOccurs

XSD provides precise control over how many times a child element can appear:

```
<xs:element name="book">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="title" type="xs:string"/>
      <!-- 1 to 5 authors -->
      <xs:element name="author" type="xs:string"
                  minOccurs="1" maxOccurs="5"/>
      <!-- Optional subtitle -->
      <xs:element name="subtitle" type="xs:string"
                  minOccurs="0"/>
      <!-- Unlimited keywords -->
      <xs:element name="keyword" type="xs:string"
                  minOccurs="0" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
</xs:element>
```

Compare this to DTD, which can only express "exactly one", "optional" (`?`), "one or more" (`+`), and "zero or more" (`*`).

Defining attributes in XSD

Attributes are declared inside complex types using `<xs:attribute>`:

```
<xs:element name="book">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="title" type="xs:string"/>
      <xs:element name="author" type="xs:string"/>
    </xs:sequence>
    <xs:attribute name="isbn" type="xs:string" use="required"/>
    <xs:attribute name="format" type="xs:string" default="paperback"/>
    <xs:attribute name="language" type="xs:string" use="optional"/>
  </xs:complexType>
</xs:element>
```

Attribute use values:

- `use="required"` — must be present (like DTD `#REQUIRED`)
- `use="optional"` — may be omitted (like DTD `#IMPLIED`)
- `use="prohibited"` — must not appear (useful when restricting inherited types)

Named types: Reusing type definitions

Instead of defining types inline (anonymous), you can name them for reuse:

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- Named complex type -->
  <xs:complexType name="addressType">
    <xs:sequence>
      <xs:element name="street" type="xs:string"/>
      <xs:element name="city" type="xs:string"/>
```

```
      <xs:element name="zip" type="xs:string"/>
      <xs:element name="country" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>

  <!-- Reuse in multiple elements -->
  <xs:element name="shippingAddress" type="addressType"/>
  <xs:element name="billingAddress" type="addressType"/>

</xs:schema>
```

XSD Restrictions, Patterns, and Type Derivation

XSD allows you to create custom types by restricting built-in types — adding constraints like value ranges, string patterns, and length limits that go far beyond what DTDs can express.

Why restrict types?

Built-in types like `xs:string` and `xs:integer` are broad. In real applications, you need tighter constraints:

- A price should be a positive number, not any decimal
- A country code should be exactly 2 uppercase letters
- An email address should match a specific pattern
- A product rating should be between 1 and 5

XSD allows you to derive new types from existing ones by adding **restrictions** (also called **facets**).

Restricting simple types

Create a new simple type by restricting a built-in type with facets:

```
<!-- A rating between 1 and 5 -->
<xs:simpleType name="ratingType">
  <xs:restriction base="xs:integer">
    <xs:minInclusive value="1"/>
    <xs:maxInclusive value="5"/>
  </xs:restriction>
</xs:simpleType>

<!-- A non-empty string of at most 100 characters -->
<xs:simpleType name="titleType">
  <xs:restriction base="xs:string">
    <xs:minLength value="1"/>
    <xs:maxLength value="100"/>
  </xs:restriction>
</xs:simpleType>

<!-- Use them in element declarations -->
<xs:element name="rating" type="ratingType"/>
<xs:element name="title" type="titleType"/>
```

Common restriction facets

Facet	Applies to	Example
`minInclusive`	Numbers, dates	`<xs:minInclusive value="0"/>`
`maxInclusive`	Numbers, dates	`<xs:maxInclusive value="100"/>`
`minLength`	Strings, lists	`<xs:minLength value="1"/>`
`maxLength`	Strings, lists	`<xs:maxLength value="255"/>`
`length`	Strings, lists	`<xs:length value="10"/>`
`pattern`	Strings	`<xs:pattern value="[A-Z]{2}"/>`
`enumeration`	Any type	`<xs:enumeration value="active"/>`
`whiteSpace`	Strings	`<xs:whiteSpace value="collapse"/>`
`totalDigits`	Decimals	`<xs:totalDigits value="5"/>`
`fractionDigits`	Decimals	`<xs:fractionDigits value="2"/>`

Enumerations: Fixed value lists

Create a type that only allows specific values (like DTD enumerated attributes, but for element content too):

```
<xs:simpleType name="statusType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="draft"/>
    <xs:enumeration value="review"/>
    <xs:enumeration value="published"/>
    <xs:enumeration value="archived"/>
  </xs:restriction>
</xs:simpleType>

<xs:element name="status" type="statusType"/>
```

Valid: `<status>draft</status>` — Invalid: `<status>pending</status>`

Patterns: Regular expression validation

The `pattern` facet uses regular expressions to constrain string values:

```
<!-- US phone number: (555) 123-4567 -->
<xs:simpleType name="phoneType">
  <xs:restriction base="xs:string">
    <xs:pattern value="\([0-9]{3}\) [0-9]{3}-[0-9]{4}"/>
  </xs:restriction>
</xs:simpleType>

<!-- ISO country code: exactly 2 uppercase letters -->
<xs:simpleType name="countryCodeType">
  <xs:restriction base="xs:string">
    <xs:pattern value="[A-Z]{2}"/>
  </xs:restriction>
</xs:simpleType>

<!-- ISBN-13: 13 digits with optional hyphens -->
<xs:simpleType name="isbnType">
  <xs:restriction base="xs:string">
    <xs:pattern value="[0-9]{3}-?[0-9]-?[0-9]{2}-?[0-9]{6}-?[0-9]"/>
  </xs:restriction>
</xs:simpleType>
```

Complex type extension: Building on existing types

You can create a new complex type by **extending** an existing one — adding new elements or attributes:

```
<!-- Base type -->
<xs:complexType name="personType">
  <xs:sequence>
    <xs:element name="firstName" type="xs:string"/>
    <xs:element name="lastName" type="xs:string"/>
  </xs:sequence>
</xs:complexType>

<!-- Extended type: adds email -->
<xs:complexType name="employeeType">
  <xs:complexContent>
    <xs:extension base="personType">
      <xs:sequence>
        <xs:element name="email" type="xs:string"/>
        <xs:element name="department" type="xs:string"/>
      </xs:sequence>
    </xs:extension>
  </xs:complexContent>
</xs:complexType>
```

This is similar to class inheritance in object-oriented programming. DITA's specialization mechanism is conceptually related — new topic types extend base types with additional constraints.

Complex type restriction: Tightening existing types

You can also restrict a complex type — keeping the same structure but tightening constraints:

```
<!-- Restricted type: lastName limited to 50 chars -->
<xs:complexType name="shortNamePersonType">
```

```
  <xs:complexContent>
    <xs:restriction base="personType">
      <xs:sequence>
        <xs:element name="firstName" type="xs:string"/>
        <xs:element name="lastName">
          <xs:simpleType>
            <xs:restriction base="xs:string">
              <xs:maxLength value="50"/>
            </xs:restriction>
          </xs:simpleType>
        </xs:element>
      </xs:sequence>
    </xs:restriction>
  </xs:complexContent>
</xs:complexType>
```

XSD Syntax Quick Reference

A compact reference for XSD element declarations, data types, facets, and common schema patterns — use this while writing and reading XSD files.

XSD schema template

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <!-- Simple type with restriction -->
  <xs:simpleType name="myType">
    <xs:restriction base="xs:string">
      <xs:maxLength value="100"/>
    </xs:restriction>
  </xs:simpleType>

  <!-- Named complex type -->
  <xs:complexType name="myComplexType">
    <xs:sequence>
      <xs:element name="child" type="xs:string"/>
    </xs:sequence>
    <xs:attribute name="id" type="xs:ID" use="required"/>
  </xs:complexType>

  <!-- Root element -->
  <xs:element name="root" type="myComplexType"/>

</xs:schema>
```

Built-in data types (most common)

Type	Description	Example value
`xs:string`	Character string	Hello
`xs:integer`	Whole number	42
`xs:decimal`	Decimal number	3.14

Type	Description	Example value
`xs:boolean`	true/false	true
`xs:date`	Calendar date	2026-04-05
`xs:dateTime`	Date and time	2026-04-05T14:30:00
`xs:time`	Time of day	14:30:00
`xs:anyURI`	URI/URL	https://example.com
`xs:float`	32-bit floating point	1.5E2
`xs:positiveInteger`	Integer > 0	1
`xs:nonNegativeInteger`	Integer >= 0	0
`xs:ID`	Unique identifier	item-001
`xs:IDREF`	Reference to an ID	item-001

Content model compositors

Compositor	Meaning	DTD equivalent
`<xs:sequence>`	Children must appear in order	`(a, b, c)`
`<xs:choice>`	Exactly one child appears	`(a | b | c)`
`<xs:all>`	Children appear in any order, each at most once	No DTD equivalent

Occurrence constraints

XSD attribute	Default	DTD equivalent
`minOccurs="0"`	1	`?` or `*`
`maxOccurs="unbounded"`	1	`*` or `+`
`minOccurs="1"` `maxOccurs="1"`	These are defaults	(none) — exactly once
`minOccurs="3"` `maxOccurs="10"`	N/A	No DTD equivalent

Restriction facets

Facet	Applies to
`minInclusive` / `maxInclusive`	Numbers, dates (value included)
`minExclusive` / `maxExclusive`	Numbers, dates (value excluded)

Facet	Applies to
`minLength` / `maxLength` / `length`	Strings
`pattern`	Strings (regex)
`enumeration`	Any type
`totalDigits` / `fractionDigits`	Decimals
`whiteSpace`	Strings (preserve/replace/collapse)

Attribute declarations

```
<xs:attribute name="id"       type="xs:ID"     use="required"/>
<xs:attribute name="lang"     type="xs:string"  use="optional"/>
<xs:attribute name="version"  type="xs:string"  fixed="1.0"/>
<xs:attribute name="status"   type="xs:string"  default="draft"/>
```

Part IV

Introduction to DITA

Topics:

What Is DITA?

DITA (Darwin Information Typing Architecture) is an XML-based standard for authoring, organizing, and publishing modular technical content — designed for reuse, conditional delivery, and multi-channel output.

What does DITA stand for?

DITA stands for **Darwin Information Typing Architecture**:

- **Darwin** — Named after Charles Darwin, because DITA uses an evolutionary model of specialization where new content types inherit from base types.
- **Information Typing** — Every topic has a specific purpose: explaining a concept, teaching a procedure, or providing reference data.
- **Architecture** — DITA is a complete system — not just a tag set, but an organizing framework with maps, keys, reuse mechanisms, and conditional processing.

DITA was originally developed at IBM in the early 2000s and became an OASIS open standard in 2005. The current version is DITA 1.3, with DITA 2.0 in development.

Core principles of DITA

DITA is built on four foundational principles:

1. **Topic-based authoring:** Content is written in self-contained topics, not long documents. Each topic addresses a single subject and can stand on its own.
2. **Information typing:** Each topic has a designated type (concept, task, reference) that determines its structure and purpose.
3. **Separation of content and presentation:** DITA topics contain only semantic markup. Formatting is applied at publish time through stylesheets and transforms.
4. **Content reuse:** Topics can be assembled into multiple publications, and content fragments can be shared across topics using conref, keyref, and other mechanisms.

How is DITA different from other XML standards?

Feature	DITA	DocBook / Custom XML
Unit of authoring	Topic (one subject per file)	Chapter or section (can be any size)
Information typing	Built-in: concept, task, reference, plus specialization	No built-in typing (some have chapter/ appendix)
Organization	Maps assemble topics into publications	Book includes chapters directly
Reuse	Conref, keyref, conkeyref, content key references	XInclude or custom mechanisms

Feature	DITA	DocBook / Custom XML
Conditional processing	Built-in with audience, platform, product attributes + DITAVAL	Custom profiling (if any)
Publishing	DITA Open Toolkit (DITA-OT) — free, extensible	Custom XSLT or commercial tools

Who uses DITA?

DITA is used by organizations that produce large volumes of technical content:

- **Software companies:** IBM, Microsoft, SAP, Oracle — user guides, API documentation, release notes
- **Hardware manufacturers:** Cisco, Nokia, Honeywell — product manuals, installation guides
- **Medical device companies:** Regulatory documentation that must be filtered by product variant and locale
- **Aerospace and defense:** Maintenance manuals with strict content reuse requirements (S1000D is a related standard)
- **Financial services:** Policy documents, compliance materials with conditional delivery by audience

The DITA ecosystem

DITA is more than a file format — it is an ecosystem of tools::

- **Authoring:** Oxygen XML Editor, Adobe FrameMaker, Arbortext, XMetaL, or any XML editor
- **Publishing:** DITA Open Toolkit (DITA-OT) — transforms DITA to HTML5, PDF, EPUB, and more
- **Content Management:** CCMS tools like SDL Tridion Docs, Vasont, Ixiasoft DITA CMS
- **Validation:** DITA DTDs, XSDs, and RELAX NG grammars for schema validation

DITA Topic Types: Concept, Task, and Reference

DITA enforces information typing through three core topic types — concept for explanations, task for procedures, and reference for lookup data — each with a specific XML structure.

Why does DITA have different topic types?

Every piece of technical content serves one of three purposes: explaining something, teaching how to do something, or providing data for lookup. DITA makes this purpose explicit by giving each type its own XML structure with specific allowed elements.

This typing matters because:

- **Authors** get a structural template that guides their writing
- **Readers** get consistent, predictable content
- **Tools** can validate, transform, and filter based on type
- **AI systems** can format responses differently for concepts vs. tasks vs. references

Concept topics: "What is...?" and "Why?"

A concept topic explains an idea, provides background, or describes architecture. It uses prose organized into sections.

Structure:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE concept PUBLIC "-//OASIS//DTD DITA Concept//EN" "concept.dtd">
<concept id="what-is-version-control">
  <title>What Is Version Control?</title>
  <shortdesc>Version control tracks changes to files over
    time, enabling collaboration and rollback.</shortdesc>
  <conbody>
    <section id="definition">
      <title>Definition</title>
      <p>Version control is a system that records changes
        to files over time.</p>
    </section>
    <section id="benefits">
      <title>Key Benefits</title>
      <ul>
        <li>Track who changed what and when</li>
        <li>Revert to previous versions</li>
        <li>Branch and merge changes</li>
      </ul>
    </section>
  </conbody>
</concept>
```

Key elements:

- `<concept>` — root element with required `id` attribute
- `<conbody>` — the body of the concept
- `<section>` — organizes content into titled subsections
- Allows: paragraphs, lists, tables, notes, code blocks, and images

Use when: Explaining what something is, why it matters, how things compare, or providing background knowledge.

Task topics: "How do I...?"

A task topic teaches a step-by-step procedure. DITA tasks have a strict structure that ensures every procedure includes prerequisites, numbered steps, and expected results.

Structure:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE task PUBLIC "-//OASIS//DTD DITA Task//EN" "task.dtd">
<task id="create-a-branch">
  <title>Creating a Git Branch</title>
  <shortdesc>Create a new branch for your feature work.</shortdesc>
  <taskbody>
    <prereq>
      <ul><li>Git installed and configured</li></ul>
    </prereq>
    <context>
      <p>Branches let you work on features without
        affecting the main code.</p>
    </context>
```

```
    <steps>
      <step>
        <cmd>Open your terminal.</cmd>
      </step>
      <step>
        <cmd>Run <codeph>git checkout -b feature-name</codeph>.</cmd>
        <stepresult>Git creates and switches to the new branch.</stepresult>
      </step>
    </steps>
    <result>
      <p>Verify with <codeph>git branch</codeph> — your
        new branch should be marked with an asterisk.</p>
    </result>
  </taskbody>
</task>
```

Key elements:

- `<taskbody>` — the task body
- `<prereq>` — what the reader needs before starting
- `<context>` — why they are doing this
- `<steps>` — numbered procedure steps
- `<step>` # `<cmd>` — each step has a command (the action)
- `<stepresult>` — what the reader should see after the step
- `<result>` — the overall outcome when all steps are done

Use when: Installation guides, how-to procedures, configuration steps, tutorials.

Reference topics: "What are the options?"

A reference topic provides structured data for lookup — parameters, options, error codes, specifications.

Structure:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE reference PUBLIC "-//OASIS//DTD DITA Reference//EN" "reference.dtd">
<reference id="git-commands">
  <title>Common Git Commands</title>
  <shortdesc>Frequently used Git commands and their purpose.</shortdesc>
  <refbody>
    <table>
      <tgroup cols="2">
        <thead>
          <row>
            <entry>Command</entry>
            <entry>Description</entry>
          </row>
        </thead>
        <tbody>
          <row>
            <entry><codeph>git init</codeph></entry>
            <entry>Initialize a new repository</entry>
          </row>
          <row>
            <entry><codeph>git clone</codeph></entry>
            <entry>Copy a remote repository</entry>
          </row>
          <row>
            <entry><codeph>git commit -m</codeph></entry>
            <entry>Save staged changes with a message</entry>
          </row>
```

```
        </tbody>
      </tgroup>
    </table>
  </refbody>
</reference>
```

Key elements:

- `<refbody>` — the reference body
- Typically contains tables, property lists, or syntax definitions
- Can use `<simpletable>` for lightweight tables or `<properties>` for name-value lists

Use when: API references, CLI options, configuration parameters, error codes, glossary entries.

Quick decision guide

When creating a new topic, ask:

- Am I explaining **what** something is or **why** it matters? # **Concept**
- Am I teaching someone **how** to do something? # **Task**
- Am I providing **facts** for someone to look up? # **Reference**

If a single source mixes all three purposes, split it into separate topics — one per purpose. This separation is how DITA achieves reusability and consistency.

DITA Maps, Reuse, and Publishing

DITA maps assemble topics into publications, reuse mechanisms eliminate duplication, and the DITA Open Toolkit transforms content into HTML5, PDF, and other formats.

What are DITA maps?

A DITA map is an XML file that organizes topics into a publication structure — like a table of contents that defines what topics appear, in what order, and in what hierarchy.

Topics are **not** embedded inside maps. Maps contain `<topicref>` elements that point to topic files. This separation means the same topic can be reused in multiple maps.

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE map PUBLIC "-//OASIS//DTD DITA Map//EN" "map.dtd">
<map id="user-guide">
  <title>Product User Guide</title>
  <topicref href="topics/overview.dita">
    <topicref href="topics/installation.dita"/>
    <topicref href="topics/configuration.dita"/>
  </topicref>
  <topicref href="topics/troubleshooting.dita"/>
  <topicref href="topics/cli-reference.dita"/>
</map>
```

Nesting `<topicref>` elements creates a hierarchy — here, "installation" and "configuration" are children of "overview."

Bookmaps: For book-like publications

A `<bookmap>` extends the basic map with book-specific structure — parts, chapters, prefaces, and appendices:

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE bookmap PUBLIC "-//OASIS//DTD DITA BookMap//EN" "bookmap.dtd">
<bookmap id="training-guide">
  <booktitle>
    <mainbooktitle>XML Fundamentals Training</mainbooktitle>
  </booktitle>
  <frontmatter>
    <topicref href="topics/preface.dita"/>
    <booklists><toc/></booklists>
  </frontmatter>
  <part navtitle="XML Basics">
    <topicref href="topics/what-is-xml.dita"/>
    <topicref href="topics/xml-syntax.dita"/>
  </part>
</bookmap>
```

Content reuse: Write once, use everywhere

DITA provides several mechanisms to reuse content without copying:

Mechanism	What it does	Use case
Topic reuse	Same topic included in multiple maps	A "Getting Started" topic appears in both the Quick Start Guide and the full User Guide
Conref	Pull content from another topic at the element level	A standard warning note reused across 20 task topics
Keyref	Indirect references resolved at build time via keys defined in the map	Product name defined once in the map, referenced everywhere via a key
Conditional processing	Include or exclude content based on attributes (audience, platform, product)	Show Windows steps for Windows users, Linux steps for Linux users

Conref example:

```
<!-- Source file: shared/warnings.dita -->
<note type="warning" id="backup-warning">
  Always back up your data before proceeding.
</note>

<!-- Reuse in any topic -->
<note conref="shared/warnings.dita#warnings/backup-warning"/>
```

Keyref example:

```
<!-- In the map -->
```

```
<keydef keys="product-name">
  <topicmeta><keywords><keyword>WidgetPro</keyword></keywords></topicmeta>
</keydef>

<!-- In any topic -->
<p>Welcome to <keyword keyref="product-name"/>!</p>
<!-- Renders as: Welcome to WidgetPro! -->
```

Conditional processing with DITAVAL

DITA elements can carry filtering attributes — `audience`, `platform`, `product`, `otherprops`. A DITAVAL file controls which values to include or exclude at build time:

```
<!-- In a topic -->
<step audience="admin">
  <cmd>Configure the firewall settings.</cmd>
</step>
<step audience="user">
  <cmd>Contact your administrator for firewall access.</cmd>
</step>

<!-- admin.ditaval -->
<val>
  <prop att="audience" val="admin" action="include"/>
  <prop att="audience" val="user" action="exclude"/>
</val>
```

This enables single-source authoring — one set of DITA files produces different outputs for different audiences.

Publishing with DITA Open Toolkit

The **DITA Open Toolkit** (DITA-OT) is the primary open-source tool for transforming DITA content into deliverable formats:

- **HTML5:** Web pages with responsive design
- **PDF:** Print-ready documents via XSL-FO
- **EPUB:** E-book format
- **Markdown:** For platforms that consume Markdown

A basic build command:

```
dita --input=userguide.ditamap --format=html5 --output=out/html
```

The toolkit reads the map, resolves all topic references, applies conditional processing (if a DITAVAL is specified), and transforms the result into the target format using XSLT stylesheets.

How XML, DTD, and XSD connect to DITA

Everything you have learned in this manual applies directly to DITA:

- **XML:** Every DITA topic and map is an XML document that must be well-formed
- **DTD:** DITA topics begin with a DOCTYPE that references DITA DTDs — the DTD enforces the allowed elements and structure for each topic type

- **XSD:** DITA also provides XSD grammars for validation in tools that prefer XSD over DTD
- **Namespaces:** XSLT transforms that process DITA use namespaces extensively

Learning XML, DTD, and XSD first gives you the foundation to understand **why** DITA works the way it does — and how to troubleshoot when things go wrong.

DITA Element Quick Reference

A compact reference for the most commonly used DITA elements across concept, task, and reference topics — use this while writing your first DITA content.

Elements common to all topic types

Element	Purpose
`<title>`	Topic title (required)
`<shortdesc>`	One-sentence summary (recommended)
`<prolog>`	Metadata container (author, dates, keywords, audience)
`<p>`	Paragraph
`<ul>` / `<ol>`	Unordered / ordered list
`<li>`	List item
`<note>`	Callout (type: note, tip, warning, important, caution)
`<codeblock>`	Multi-line code listing
`<codeph>`	Inline code phrase
`<b>` / `<i>`	Bold / italic emphasis
`<xref>`	Cross-reference to another topic or external URL
`<image>`	Image reference
`<section>`	Titled subsection within a topic body
`<table>`	Full table (with tgroup, thead, tbody)
`<simpletable>`	Lightweight table (simpler markup)
`<fig>`	Figure with title and image

Concept-specific elements

Element	Purpose
`<concept>`	Root element (DOCTYPE: concept.dtd)

Element	Purpose
`<conbody>`	Body of the concept topic

Task-specific elements

Element	Purpose
`<task>`	Root element (DOCTYPE: task.dtd)
`<taskbody>`	Body of the task topic
`<prereq>`	Prerequisites before starting
`<context>`	Background for why the task is needed
`<steps>`	Container for numbered steps
`<step>`	A single step in the procedure
`<cmd>`	The imperative action in a step
`<info>`	Additional detail for a step
`<substeps>`	Sub-procedure within a step
`<stepresult>`	Expected outcome of a single step
`<result>`	Overall outcome when all steps are done

Reference-specific elements

Element	Purpose
`<reference>`	Root element (DOCTYPE: reference.dtd)
`<refbody>`	Body of the reference topic
`<properties>`	Property table (name-type-value)
`<refsyn>`	Syntax definition

Map elements

Element	Purpose
`<map>`	Basic map root element
`<bookmap>`	Book-structured map root element
`<topicref>`	Reference to a topic file
`<mapref>`	Reference to another map (submap)
`<keydef>`	Key definition for indirect references

Element	Purpose
`<topicmeta>`	Metadata for a topicref

DOCTYPE declarations

```
<!-- Concept -->
<!DOCTYPE concept PUBLIC "-//OASIS//DTD DITA Concept//EN" "concept.dtd">

<!-- Task -->
<!DOCTYPE task PUBLIC "-//OASIS//DTD DITA Task//EN" "task.dtd">

<!-- Reference -->
<!DOCTYPE reference PUBLIC "-//OASIS//DTD DITA Reference//EN" "reference.dtd">

<!-- Map -->
<!DOCTYPE map PUBLIC "-//OASIS//DTD DITA Map//EN" "map.dtd">

<!-- Bookmap -->
<!DOCTYPE bookmap PUBLIC "-//OASIS//DTD DITA BookMap//EN" "bookmap.dtd">
```

Appendix A

Learner Exercises

Topics:

Exercise 1: Writing Well-Formed XML Documents

Practice XML syntax rules by creating well-formed XML documents from scratch, fixing broken documents, and understanding the XML tree structure.

- A text editor with XML support (VS Code with the XML extension, or any XML-aware editor)
- Read Part 1: Chapters 1-4 (What Is XML, Syntax Rules, Elements and Attributes, Namespaces)

In this exercise, you will practice core XML skills by creating documents for a fictional online library system called **BookVault**. You will write XML from scratch, identify and fix syntax errors, and explore the document tree.

Create a working folder: `xml-exercises/`

1. Create a well-formed XML document: `xml-exercises/catalog.xml`

   Requirements:

   - Include the XML declaration with version and encoding
   - Root element: `<catalog>`
   - Include at least 3 `<book>` child elements
   - Each book must have: `<title>`, `<author>`, `<year>`, and `<price>` children
   - Each `<book>` must have a `category` attribute (fiction, non-fiction, or technical)
   - Each `<price>` must have a `currency` attribute
   - Include at least one XML comment

   Open the file in your XML editor — it should show no syntax errors (green/valid indicator).

2. Create a file `xml-exercises/broken.xml` with the following content, then find and fix all 6 errors:

```
<?xml version="1.0" encoding="UTF-8"?>

<Library>
  <book category=fiction>
    <Title>The Great Adventure</title>
    <author>Jane & John Doe</author>
    <year>2024
    <price currency="USD">14.99</price>
  </book>
  <book category="technical">
    <title>Learning XML</title>
    <author>Bob Smith</author>
    <year>2025</year>
    <price currency="USD">29.95</price>
  </book>
</library>
```

   Hint: The errors involve case sensitivity, quoting, closing tags, and entity references. Write down each error and its fix.

   After fixing all errors, the document should pass well-formedness checking in your XML editor.

3. Create a document with mixed content: `xml-exercises/article.xml`

   Requirements:

   - Root element: `<article>`

- Include a `<title>` and at least 2 `<paragraph>` elements
- Paragraphs must contain mixed content: text mixed with `<bold>`, `<italic>`, and `<code>` inline elements
- Include one `<code-listing>` element that uses a CDATA section to contain XML code as text
- Use at least two entity references (`&lt;`, `&amp;`, etc.) where appropriate

4. Create a document with namespaces: `xml-exercises/report.xml`

   Requirements:

   - Root element: `<report>` with a default namespace of `http://example.com/reports`
   - Add a prefixed namespace `meta:` for `http://example.com/metadata`
   - Include `<title>` and `<content>` in the default namespace
   - Include `<meta:author>` and `<meta:date>` in the metadata namespace

5. Analyze the tree structure of your `catalog.xml` file.

   Write down (on paper or in a text file):

   - What is the root element?
   - What are the children of the root?
   - What are the children of the first `<book>`?
   - Which elements are siblings?
   - What is the parent of `<price>`?
   - Name all ancestors of `<author>`

> **Tip**
> Understanding the tree structure is critical for XPath (used in XSLT and XSD) and for navigating DITA topic structures.

Self-Check — you should now be able to:

- # Write well-formed XML documents from scratch
- # Identify and fix common XML syntax errors
- # Use entity references for special characters
- # Write mixed content with inline markup
- # Use CDATA sections for verbatim text
- # Declare and use XML namespaces
- # Describe parent, child, sibling, and ancestor relationships in an XML tree

Exercise 2: Writing DTDs and Validating XML

Practice writing DTD element and attribute declarations, connecting DTDs to XML documents, and using validation to catch structural errors.

- Completed Exercise 1 (XML Basics)
- Read Part 2: Chapters 5-7 (What Is DTD, Element Declarations, Attributes and Entities)

- An XML editor that supports DTD validation (VS Code with XML extension, or Oxygen XML)

You will create DTDs for BookVault's XML documents and experience how schema validation catches errors that well-formedness checking alone misses.

1. Create an external DTD file: `xml-exercises/catalog.dtd`

   Requirements:

   - Declare a `<catalog>` element containing one or more `<book>` elements
   - Declare `<book>` with children: `title`, `author+`, `year`, `price`, `description?` (in that order)
   - Declare all child elements as `#PCDATA`
   - Declare `<description>` with mixed content: text, `<bold>`, and `<italic>`
   - Add attributes to `<book>`: `isbn` (CDATA, required), `category` (enumeration: fiction|non-fiction|technical, required)
   - Add a `currency` attribute to `<price>` (CDATA, default "USD")

   Your DTD file should be syntactically correct.

2. Update your `catalog.xml` to reference the DTD, then validate it.

   Add a DOCTYPE declaration after the XML declaration:

   ```
   <!DOCTYPE catalog SYSTEM "catalog.dtd">
   ```

   Make sure your XML document conforms to the DTD rules — add `isbn` attributes, ensure correct element order, etc.

   Your editor should report: "Document is valid" (no validation errors).

3. Deliberately break validation to understand error messages.

   Try each of these changes one at a time and observe the error message:

   a. Remove a required `isbn` attribute — what error does the validator report?

   b. Put `<author>` before `<title>` — what error?

   c. Add an element not declared in the DTD (e.g., `<rating>`) — what error?

   d. Use a category value not in the enumeration (e.g., `category="mystery"`) — what error?

   Revert each change before trying the next.

> **Tip**
> Learning to read validation error messages is one of the most valuable skills in XML work. The error usually tells you exactly what is wrong and where.

4. Add entity declarations to your DTD and use them in your XML.

   Requirements:

   - Define a general entity `publisher` with value "BookVault Publishing"
   - Define a general entity `copyright-notice` with a copyright message
   - Add a `<publisher>` element to the DTD and use the entity in your XML
   - (Optional) Define a parameter entity for the inline content model `(#PCDATA | bold | italic)*` and apply it to multiple elements

5. Create a new file `xml-exercises/note.xml` with an internal DTD.

Requirements:

- Define the DTD inline inside the XML document
- Elements: `<note>` containing `<to>`, `<from>`, `<subject>`, `<body>`
- Add a `priority` attribute to `<note>` with enumerated values: low, normal, high (default: normal)
- Write a sample note and validate it

Self-Check — you should now be able to:

- # Write element declarations with sequences, choices, and occurrence indicators
- # Write attribute declarations with types and default behaviors
- # Create both internal and external DTDs
- # Connect an XML document to its DTD via DOCTYPE
- # Validate XML against a DTD and interpret error messages
- # Define and use general entities
- # Explain the difference between well-formed and valid

Exercise 3: Writing XML Schemas (XSD)

Practice writing XSD schemas with data types, restrictions, and complex types — then compare XSD validation with DTD validation to understand the differences.

- Completed Exercise 2 (DTD Validation)
- Read Part 3: Chapters 8-10 (What Is XSD, Elements and Types, Restrictions and Patterns)
- An XML editor that supports XSD validation

You will re-create the BookVault catalog schema using XSD — experiencing the additional power XSD provides over DTDs. Then you will write schemas for new document types.

1. Create an XSD schema: `xml-exercises/catalog.xsd`

   Requirements:

   - Root element: `catalog` containing one or more `book` elements
   - Each `book` has children: `title` (string), `author` (string, 1 to 5 occurrences), `year` (integer), `price` (decimal), optional `description` (string)
   - `book` has attributes: `isbn` (string, required), `category` (enumeration: fiction/non-fiction/technical, required)
   - `price` has attribute: `currency` (string, default "USD")

   Your XSD should be a well-formed XML document with `<xs:schema>` as the root.
2. Create `xml-exercises/catalog-xsd.xml` that validates against your XSD.

   Use `xsi:noNamespaceSchemaLocation` to link to your schema:

```
<catalog xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:noNamespaceSchemaLocation="catalog.xsd">
```

Include at least 3 books with correct data types in all elements.

The document should validate without errors.

3. Test XSD data type validation by introducing type errors.

   Try each change and observe what happens:

   a. Put text in the `<year>` element (declared as integer): `<year>twenty twenty-five</year>`
   b. Put a negative number in `<price>` — does it validate? (It should, because you declared it as `xs:decimal`, not a restricted type)
   c. Add a 6th `<author>` element (exceeds maxOccurs="5")

> **Tip**
> Notice how XSD catches data type errors that a DTD would miss. A DTD treats all content as text, so `<year>hello</year>` would be valid against a DTD but invalid against an XSD.

4. Enhance your XSD with custom restricted types.

   Add these custom types:

   - `priceType` — decimal with minimum value of 0.00 and maximum of 9999.99, with exactly 2 fraction digits
   - `yearType` — integer between 1900 and 2100
   - `isbnType` — string matching the pattern `[0-9]{3}-[0-9]-[0-9]{2}-[0-9]{6}-[0-9]`

   Replace the built-in types with your custom types and re-validate your XML.

   Now a `<year>1800</year>` or `<price>-5.00</price>` should fail validation.

5. Create a new schema for a different domain: `xml-exercises/employees.xsd`

   Requirements:

   - Define a named complex type `addressType` with: `street`, `city`, `state`, `zip`
   - Define a named complex type `personType` with: `firstName`, `lastName`, `email` (anyURI type)
   - Define `employeeType` by extending `personType` with: `department`, `hireDate` (date), `address` (addressType)
   - Root element: `company` with attribute `name`, containing one or more `employee` elements

   Create a matching `xml-exercises/employees.xml` with sample data.

6. Reflect on DTD vs. XSD differences.

   Write down answers to these questions:

   - What validation errors did XSD catch that DTD could not? Give specific examples.
   - Which schema language was easier to write? Why?
   - When would you choose DTD over XSD?
   - Why does DITA provide both DTD and XSD grammars?

Self-Check — you should now be able to:

- # Write XSD schemas with simple and complex types
- # Use built-in data types: string, integer, decimal, date, boolean
- # Create custom types with restrictions: enumerations, patterns, ranges

- # Control element occurrences with minOccurs/maxOccurs
- # Define named complex types and reuse them
- # Extend complex types (type inheritance)
- # Link XML documents to XSD schemas and validate
- # Explain the key differences between DTD and XSD

Exercise 4: Your First DITA Project

Apply everything you have learned by creating a complete DITA mini-project — concept, task, and reference topics assembled into a map and built into HTML5 output.

- Completed Exercises 1-3
- Read Part 4: Chapters 11-13 (What Is DITA, Topic Types, Maps and Publishing)
- DITA Open Toolkit (DITA-OT) installed — download from `https://www.dita-ot.org`
- Java 17 or later installed (required by DITA-OT)

You will create documentation for BookVault — the fictional online library system from the earlier exercises. This time, you will write in DITA format using proper topic types, assemble topics into a map, and build HTML5 output.

Create a project folder: `bookvault-docs/`

1. Create a concept topic: `bookvault-docs/what-is-bookvault.dita`

   Requirements:

   - DOCTYPE: `<!DOCTYPE concept PUBLIC "-//OASIS//DTD DITA Concept//EN" "concept.dtd">`
   - Include `<title>` and `<shortdesc>`
   - Include at least two `<section>` elements with titles inside `<conbody>`
   - Use `<ul>` for a feature list
   - Include a `<table>` comparing BookVault to a traditional library
   - Add one `<note type="tip">`

   The file should be well-formed XML (check in your editor).
2. Create a task topic: `bookvault-docs/creating-an-account.dita`

   Requirements:

   - DOCTYPE: task.dtd
   - Include `<prereq>` listing requirements (email, age 13+)
   - Include `<context>` explaining why an account is needed
   - Write 5-7 `<step>` elements for the registration process
   - At least one step must have `<substeps>` (e.g., for configuring profile settings)
   - At least two steps must have `<stepresult>`
   - Include a `<result>` section with a verification check
3. Create a reference topic: `bookvault-docs/search-options.dita`

   Requirements:

- DOCTYPE: reference.dtd
- Include a `<table>` with columns: Search Field, Syntax, Example, Description
- Document at least 6 search options (title, author, ISBN, category, year, keyword)
- Add a `<section>` with `<codeblock>` showing example search queries

4. Create a DITA map: `bookvault-docs/bookvault-guide.ditamap`

   Requirements:

   - DOCTYPE: map.dtd
   - Include a `<title>`
   - Add `<topicref>` elements for all three topics in logical order
   - Nest the task and reference topics under the concept topic to create a hierarchy

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE map PUBLIC "-//OASIS//DTD DITA Map//EN" "map.dtd">
<map id="bookvault-guide">
  <title>BookVault User Guide</title>
  <topicref href="what-is-bookvault.dita">
    <topicref href="creating-an-account.dita"/>
    <topicref href="search-options.dita"/>
  </topicref>
</map>
```

5. Build your DITA project to HTML5.

```
dita --input=bookvault-docs/bookvault-guide.ditamap \
     --format=html5 \
     --output=out/bookvault
```

   If DITA-OT is not in your PATH, use the full path to the `dita` command.

   The build should complete without errors. Open `out/bookvault/index.html` in your browser and verify:

   - Table of contents appears based on your map structure
   - Concept topic renders with sections, lists, and tables
   - Task topic renders with numbered steps and substeps
   - Reference topic renders with a formatted table

6. Reflect on the connection between XML, schemas, and DITA.

   Answer these questions:

   - Open one of your DITA files — where is the **DOCTYPE** declaration? What DTD does it reference?
   - What happens if you put a `<steps>` element inside a concept topic? (Try it!) Why does validation fail?
   - How does the DITA DTD enforce information typing? (Hint: the `concept.dtd` only allows `<conbody>`, not `<taskbody>`.)
   - Where do XML elements, attributes, and tree structure appear in DITA?
   - How does the map use the XML tree (parent-child nesting) to create the TOC hierarchy?

Important

> This is the key insight of this training: DITA is XML with a purpose-built schema. Every XML concept you learned — elements, attributes, trees, DTDs, validation — is the foundation that makes DITA work.

Self-Check — you should now be able to:

- # Create valid DITA concept, task, and reference topics from scratch
- # Assemble topics into a DITA map
- # Build HTML5 output using DITA-OT
- # Explain how DITA uses XML, DTDs, and validation
- # Understand why topic typing enforces structure
- # Navigate and verify DITA output in a browser

Next Steps: To continue your DITA learning journey, explore the **Extense DITA Training Guide** — which covers information architecture, content reuse, metadata enrichment, publishing automation, and advanced development topics.

EXTENSE LLC

Technology Consulting & Training Solutions

www.extense.com